

INGEGNERIA DEL SOFTWARE · INGEGNERIA
INFORMATICA · UNISA

Introduzione all'Ingegneria del Software

Appunti di ripasso e materiale d'esame — Lezione 1

Requisiti & vincoli

Dal programma al sistema

Crisi del software

Legge di Brooks

Chaos Report

Fonte del corso

Slide "Lezione 1 — Introduzione alla Ingegneria del Software", P. Ritrovato — N. Capuano, DIEM, Università di Salerno.

Testo di riferimento: I. Sommerville, *Ingegneria del Software*, 10^a ed., Pearson, Capitolo 1.

Rielaborato e ampliato per lo studio e la preparazione dell'esame.

Indice

1. Da un programma a un sistema

Un problema apparentemente semplice · Requisiti funzionali e non funzionali · Vincoli di progettazione · Il ciclo comprensione→progettazione→implementazione→test · Processo ideale vs realtà · Stima dell'effort

2. Costruire un sistema

Programma vs sistema · Complessità e i tre modi per gestirla · Processo di produzione del software · Il problema della comunicazione tra le persone

3. Ingegneria del software: perché serve

La crisi del software (NATO 1968) · Cause dei fallimenti · Il Chaos Report (1995 e 2014) · Lo studio di Capers Jones · Problemi non tecnici · La legge di Brooks · I "cowboy programmers" · Definizione di Ingegneria del Software

4. Riepilogo per l'esame

Mappa concettuale in breve · Domande di autoverifica con risposta

1 · Da un programma a un sistema

1.1 Un problema che sembra semplice

Il punto di partenza della lezione è volutamente banale: **"data una collezione di stringhe contenute in un file, ordinarle alfabeticamente e scriverle in un altro file"**. L'obiettivo didattico è mostrare che anche un compito che sembra risolvibile in dieci minuti nasconde in realtà una quantità di decisioni che vanno prese *prima* di scrivere una sola riga di codice.

Se ci si ferma a riflettere, emergono subito delle domande a cui l'enunciato non risponde:

- **Formato di input/output** — le stringhe sono su righe separate? in un CSV? in JSON?
- **Criterio di ordinamento** — maiuscole e minuscole si trattano allo stesso modo? Come si ordinano le lettere accentate? I caratteri non alfabetici vanno ignorati o considerati?
- **Errori e casi speciali** — cosa succede se il file di output esiste già? Lo sovrascrivo, chiedo conferma, genero un errore?
- **Performance** — quanta memoria posso usare? È accettabile un algoritmo con complessità quadratica $O(n^2)$, o servono liste molto grandi che richiedono un algoritmo $O(n \log n)$?
- **Usabilità** — serve un'interfaccia grafica? Il programma deve guidare l'utente passo-passo o basta una riga di comando?

IDEA CHIAVE Un enunciato di "problema" non è mai una specifica completa. Il lavoro dell'ingegnere del software comincia proprio nel trasformare un'esigenza vaga in un insieme di **requisiti** precisi.

1.2 Requisiti: funzionali e non funzionali

Le domande della sezione precedente si possono raggruppare in due grandi famiglie:

Tipo	Risponde a...	Come si misura	Esempio dal caso "ordinamento"
Requisiti funzionali	Cosa deve fare il programma	Booleano: soddisfatto / non soddisfatto	"Il programma legge un file di testo e produce un file con le stringhe ordinate alfabeticamente"
Requisiti non funzionali	Come devono essere realizzate le funzionalità	Scala continua: più o meno soddisfatto	"Il tempo di risposta per 1 milione di stringhe deve essere inferiore a 2 secondi"

La distinzione nella misura non è un dettaglio formale: un requisito funzionale o è implementato o non lo è; un requisito non funzionale (tempo di risposta, usabilità,

sicurezza...) può essere soddisfatto "abbastanza bene" o "meglio della versione precedente", ed è quindi oggetto di compromessi (*trade-off*) continui durante il progetto.

DA RICORDARE I requisiti descrivono **il problema reale** da risolvere, non ancora la soluzione. Servono ad assicurarsi che quello che verrà costruito sia davvero utile a chi lo userà. Spesso sono **negoziabili**: si distingue tra caratteristiche *necessarie* e caratteristiche solo *desiderabili*, e questa distinzione guida le priorità quando tempo e budget sono limitati.

1.3 Vincoli di progettazione (Design Constraint)

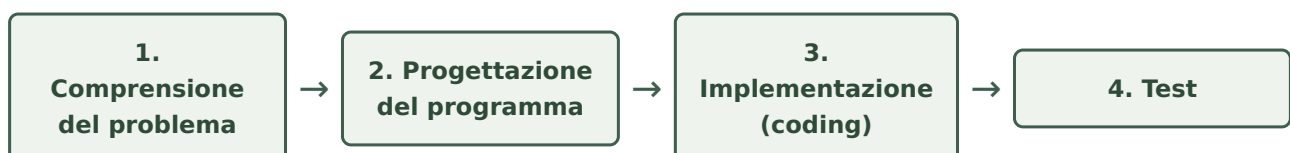
Oltre ai requisiti, ci sono condizioni che **limitano dall'esterno** le scelte di chi progetta il software, indipendentemente da cosa il software debba fare. Sono spesso trattati come un caso particolare di requisito non funzionale, perché anche loro non riguardano "cosa" ma "come/dove/con cosa" il software va realizzato.

- **Piattaforma** — Windows, Linux, macOS, Android?
- **Tecnologie obbligate** — es. deve usare Java e PostgreSQL perché l'azienda ha già competenze/licenze su quello stack
- **Integrazione con sistemi esistenti** — es. deve poter esportare dati verso Excel
- **Dimensione dei dati** — dimensione media attesa e dimensione massima da supportare
- **Tempi di realizzazione** — scadenza per una versione "alpha", scadenza per la versione finale

ATTENZIONE, PUNTO SOTTILE Non tutte le caratteristiche tecniche sono vincoli imposti dall'esterno: alcune sono realmente **vincoli** (non negoziabili, imposti dal cliente o dal contesto), altre sono invece **scelte di progettazione** fatte liberamente dal team (es. "uso un database relazionale" può essere una scelta interna, non un vincolo, se nessuno l'ha imposta dall'esterno). Riconoscere la differenza è importante: un vincolo va rispettato sempre, una scelta di progettazione si può rivedere se emergono problemi.

1.4 Il processo per arrivare al codice

Una volta chiariti requisiti e vincoli, il percorso "ideale" per costruire il software si articola in quattro fasi.



Fase	Cosa si fa
1. Comprensione	Si raccolgono e chiariscono requisiti (funzionali e non) e design constraint. È la fase in cui si risponde alle domande viste sopra.

2. Progettazione	Si identificano i componenti del sistema (funzioni, classi, moduli, database...), si definisce come interagiscono tra loro, si progetta l'interfaccia utente, le interfacce verso altri sistemi e si scelgono gli algoritmi — sempre rispettando requisiti e vincoli definiti prima.
3. Implementazione	Le decisioni prese in fase di progettazione vengono tradotte in codice sorgente vero e proprio.
4. Test	Si verifica il comportamento del programma su input noti (<i>casi di test</i>), scelti in modo da coprire bene le situazioni possibili. Se il risultato non è quello atteso si procede con <i>debug</i> (individuare la causa), <i>fix</i> (correggere) e <i>re-test</i> , fino a quando tutti i casi passano.

Processo ideale vs quello che succede davvero

Sulla carta le quattro fasi sono **sequenziali**: prima capisco, poi progetto, poi scrivo codice, poi testo. Nella pratica il flusso è molto più **iterativo**: un problema emerso in fase di test spesso costringe a tornare indietro alla progettazione o persino a rivedere la comprensione del problema (magari un requisito era stato frainteso); un dettaglio scoperto durante l'implementazione può rendere necessario ridisegnare un componente. Le frecce nel diagramma reale vanno quindi **in entrambe le direzioni** tra tutte le fasi, non solo in avanti.

PERCHÉ È IMPORTANTE SAPERLO Questo scarto tra "processo ideale" (lineare) e "quello che succede" (iterativo, con andirivieni) è il seme concettuale da cui nascono, più avanti nel corso, i diversi *modelli di processo software* (a cascata, incrementale, agile...): ognuno gestisce in modo diverso il fatto che tornare indietro è quasi sempre necessario.

1.5 Stima dell'effort

Un aspetto centrale della gestione di un progetto software è stimare **quanto lavoro (effort)** servirà, di solito misurato in ore o giorni-persona. Prendendo ancora l'esempio del programma di ordinamento (scritto da soli, con un'interfaccia minimale a riga di comando per inserire nome del file di input e di output), stimare "a occhio" il tempo totale è difficile e poco affidabile.

La tecnica proposta per migliorare la stima è il **project breakdown** (scomposizione del progetto): si divide il lavoro in compiti più piccoli e si stima l'effort di ciascuno separatamente, poi si sommano le stime parziali. Le stime sui compiti piccoli sono più facili da fare correttamente di una stima "globale" a spanne.

Task	Effort (ore)	Giorno inizio	Giorno fine
Comprendere il problema	...	...	...
Progettare il programma	...	...	...
Coding — algoritmo di ordinamento	...	...	...
Coding — lettura/scrittura file	...	...	...

Coding — interfaccia utente	...	...	...
Test	...	...	...
Totale	Σ		

APPROFONDIMENTO Questa tecnica è la versione elementare di quella che nella gestione progetti si chiama **Work Breakdown Structure (WBS)**: scomporre un progetto grande in unità di lavoro piccole e stimabili è alla base di qualunque pianificazione, non solo in informatica.

2 · Costruire un sistema

2.1 Dai programmi ai sistemi

Il caso dell'ordinamento di stringhe è un *programma*: piccolo, con un solo scopo, realizzabile da una persona sola. Un **sistema software** reale è qualitativamente diverso, non solo "più grande":

Programma	Sistema
Poche funzionalità	Molte funzionalità, ciascuna con più caratteristiche
Un'unica interfaccia (es. riga di comando)	Varietà di interfacce, sia esterne (utenti, altri sistemi) sia interne (tra i moduli)
Un solo tipo di utente	Più utenti, con ruoli e permessi diversi
Dati semplici	Grandi volumi di dati, tipi e strutture dati diverse
Nessuna integrazione	Collegamenti e connessioni con altri sistemi
Realizzato da un programmatore singolo	Realizzato da un team di sviluppo , anche di grandi dimensioni

L'ultima riga è la conseguenza più importante di tutte le altre: appena il problema cresce in dimensione e complessità, cresce anche l'effort necessario e, di conseguenza, la dimensione e la complessità del software da realizzare — e a un certo punto una sola persona non basta più.

2.2 Tre modi per gestire la complessità

Quando un problema diventa troppo grande per essere affrontato "a mano", esistono tre leve — complementari, non alternative — per tenerlo sotto controllo.

a) Divide et impera

- **Decomposizione del problema:** spezzare un problema grande in sotto-problemi più piccoli e più facili da risolvere singolarmente
- **Modularizzazione della soluzione:** costruire il sistema come insieme di moduli/componenti ben distinti
- **Separazione delle responsabilità:** ogni modulo/persona si occupa di una parte precisa, senza sovrapposizioni confuse
- **Iterazioni incrementali:** costruire il sistema a piccoli passi successivi invece che tutto insieme

b) Usare strumenti migliori

- Database (per gestire grandi quantità di dati strutturati)

- IDE (ambienti di sviluppo integrati, che velocizzano scrittura e debug del codice)
- Documentazione automatica
- Test automatizzati
- Sistemi di controllo di versione (es. Git)

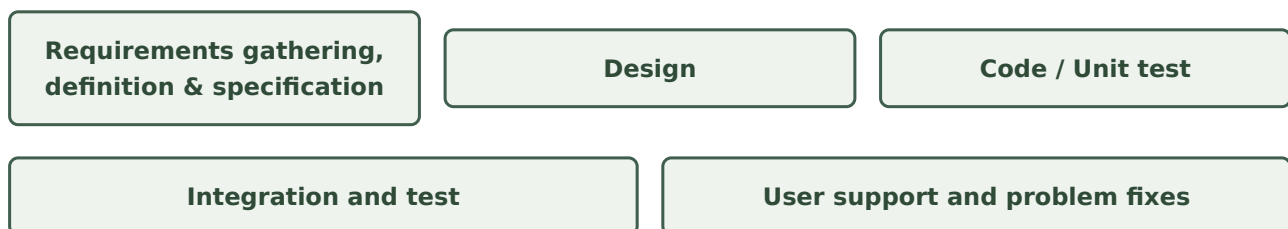
c) Migliorare processo e metodologia

Serve a **coordinare** il lavoro di persone che svolgono compiti collegati tra loro, facendo il salto da "un gruppo di individui" a un vero **lavoro di squadra**. È la leva meno "tecnica" delle tre, ma è quella che — come si vedrà nella sezione 3 — nella pratica fa più spesso la differenza tra successo e fallimento di un progetto.

2.3 Il processo di produzione del software

DEFINIZIONE Il **processo di produzione del software** è l'insieme delle attività necessarie per costruire un sistema software.

Esempio di attività tipiche (che ritroveremo più nel dettaglio nelle prossime lezioni):



...tutte coordinate dal **Software project management** (planning, organizing, measuring, adjusting)

Ma elencare le attività non basta. Un processo, per essere davvero definito, deve specificare anche:

- La **sequenza** in cui le attività si susseguono
- Gli **input** e gli **output** di ciascuna attività
- I **partecipanti** di ciascuna attività
- Le **precondizioni** per stabilire quando un'attività può iniziare
- Le **postcondizioni** per stabilire quando un'attività può dirsi completata

2.4 Il problema della comunicazione

Più il progetto è complesso, più risorse (in particolare persone) servono. Ma aggiungere persone ha un costo nascosto: **più persone significa più percorsi di comunicazione possibili**, e quindi più occasioni di incomprensioni ed errori.

FORMULA Per **n** persone nel team, il numero di percorsi di comunicazione potenziali a due a due è:

$$n \cdot (n - 1) / 2$$

Persone (n)	Percorsi di comunicazione
2	1
4	6
6	15
10	45

Il numero di percorsi cresce con il **quadrato** del numero di persone, non in proporzione lineare: raddoppiare il team più che raddoppia il rischio di incomprensioni. Da qui nasce la necessità di **strutture organizzative** (sotto-team, referenti, gerarchie di comunicazione, riunioni strutturate) che riducano il numero di canali diretti necessari, sostituendoli con canali "a imbuto" più gestibili.

3 · Perché serve l'Ingegneria del Software

3.1 La "crisi del software"

Il termine nasce alla **NATO Software Engineering Conference del 1968**, il primo grande incontro internazionale in cui l'industria riconobbe pubblicamente di avere un problema strutturale nello sviluppo software. I sintomi individuati:

- Progetti completati fuori budget e fuori tempo (quando venivano completati)
- Qualità del software in costante peggioramento
- Software che, una volta consegnato, non risolveva il vero problema per cui era stato commissionato
- Costi di manutenzione sempre più alti nel tempo

La conclusione della conferenza fu che lo sviluppo software era una disciplina **troppo giovane e immatura**, e che serviva un **approccio sistematico**, con basi solide come le altre discipline ingegneristiche — da qui il nome "Ingegneria del Software", coniato proprio in quell'occasione.

3.2 Cause dei fallimenti

Una prima causa evidente è l'aumento della complessità dei sistemi richiesti, via via che i computer sono diventati più potenti e diffusi:

"Fintanto che non c'erano i computer, la programmazione non era un problema. Quando i computer erano pochi e di capacità limitata, la programmazione divenne un lieve problema. Ora che i nostri computer hanno capacità enormi, la programmazione è diventata un problema altrettanto enorme."

— E. Dijkstra, 1972

Ma, come mostrano gli studi seguenti, la complessità tecnica non è affatto l'unica causa dei fallimenti.

3.3 Il Chaos Report

Studio periodico dello **Standish Group** sull'esito di progetti software di grandi dimensioni.

Edizione 1995

Su 300 progetti analizzati, solo il **16%** fu completato nei tempi e nei costi previsti. Le cause principali di fallimento individuate:

- Mancata comunicazione con gli utenti
- Requisiti e specifiche incompleti
- Requisiti e specifiche modificati in corso d'opera

Edizione 2014

Esito	Percentuale
Completati con successo	39%
Problematici (in ritardo, oltre budget o con funzionalità ridotte)	43%
Falliti (annullati o mai realmente usati)	18%

Progetti di successo — attributi comuni

- Coinvolgimento degli utenti
- Supporto dei vertici aziendali
- Requisiti chiari
- Pianificazione adeguata

Progetti problematici — attributi comuni

- Mancanza di comunicazione con gli utenti
- Requisiti e specifiche incomplete
- Requisiti modificati durante il progetto

Progetti annullati — attributi comuni

- Requisiti incompleti
- Mancanza di coinvolgimento degli utenti
- Mancanza di risorse

FILO CONDUTTORE DA NOTARE In tutte e tre le categorie (successo, problematici, falliti) il fattore che ricorre di più non è mai "il codice era scritto male": è quasi sempre legato a **requisiti** e a **comunicazione con gli utenti**. È lo stesso filo conduttore che si ritroverà nello studio di Capers Jones e nella legge di Brooks.

3.4 Lo studio di Capers Jones (2008)

Analizza dove hanno origine gli errori nei progetti software:

Origine dell'errore	Percentuale
Errori di codifica	38,33%
Errori di progettazione	24,17%
Errori di documentazione	13,33%
Errori nei requisiti	12,50%
Correzione errata di problemi precedenti	11,67%

La domanda che pone la slide originale è volutamente provocatoria: **è più preoccupante un errore di codifica o un errore nei requisiti**, anche se numericamente gli errori di codifica sono i più frequenti?

APPROFONDIMENTO — PERCHÉ I REQUISITI "PESANO" DI PIÙ La risposta, sviluppata a fondo più avanti nel corso (concetto noto come *costo del cambiamento*, formalizzato da B. Boehm), è che il **costo di correggere un errore cresce man mano che si scopre più tardi nel ciclo di vita del progetto**. Un errore nei requisiti scoperto già in fase di requisiti costa poco da correggere; lo stesso errore, se scoperto solo dopo che il sistema è stato progettato, implementato e magari già consegnato, può costringere a rifare design, codice, test e documentazione: il costo può aumentare di un ordine di grandezza (o più) rispetto a un errore di codifica corretto subito durante lo unit test. Ecco perché, pur essendo meno frequenti in percentuale, gli errori nei requisiti sono spesso i più pericolosi e costosi.

3.5 I problemi "non tecnici"

Al crescere di dimensione e complessità dei progetti, i problemi che emergono con più forza non sono quasi mai di natura puramente tecnica, ma riguardano la mancanza di:

- Impegno reale dei vertici aziendali verso il progetto
- Pianificazione appropriata delle attività
- Forza lavoro con competenze ed esperienza adeguate
- Monitoraggio continuo dell'andamento del progetto
- Disponibilità del team ad accettare cambiamenti nel modo di lavorare

3.6 La legge di Brooks

"Aggiungere manodopera a un progetto software in ritardo lo renderà ancora più in ritardo."

— F. Brooks, 1975 (*The Mythical Man-Month*)

L'origine dell'errore è un'assunzione implicita molto diffusa nella gestione dei progetti: si misura il costo in **mesi-persona**, come se tempo e persone fossero perfettamente intercambiabili. Un progetto da 100 mesi-uomo, secondo questa logica, potrebbe essere completato in 10 mesi da 10 persone, in 4 mesi da 25 persone, in 20 mesi da 5 persone, e così via — bastano prodotti aritmetici.

Nella pratica questa assunzione è falsa per due motivi principali:

- **Non tutte le attività si possono dividere** tra un numero arbitrario di persone (alcune parti del lavoro sono intrinsecamente sequenziali, o richiedono che sia la stessa persona a portarle avanti per coerenza)
- **Più persone → più comunicazione e coordinamento necessari** (si veda la formula $n(n-1)/2$ della sezione 2.4): oltre un certo numero, il tempo "consumato" a spiegare il contesto ai nuovi arrivati e a coordinare il lavoro supera il tempo che i nuovi arrivati fanno effettivamente risparmiare.

Il risultato è la curva "a U": aggiungere qualche persona in più a un progetto in ritardo aiuta fino a un certo punto, ma oltre una certa soglia **ogni persona aggiuntiva diventa una**

perdita netta di tempo per l'intero team, perché il costo di comunicazione supera il contributo produttivo della nuova risorsa.

CONCLUSIONE DELLA LEGGE DI BROOKS Il fallimento di un progetto software è spesso dovuto alla **cattiva gestione di aspetti non tecnici** — in questo caso, organizzativi — più che a carenze tecniche vere e proprie.

3.7 Mancanza di disciplina: i "cowboy programmers"

Un ultimo fattore di rischio, questa volta legato all'atteggiamento individuale più che all'organizzazione, è la mancanza di disciplina nello scrivere software. Alcuni atteggiamenti tipici del "cowboy programmer" (da evitare assolutamente in un lavoro di squadra):

- "Se hai bisogno di commenti per capire il mio codice, sei troppo stupido per leggerlo"
- "Il mio codice è perfetto, quindi perché dovrei testarlo?"
- "Il mio codice è perfetto, quindi perché dovrei aggiornarlo?"
- "Se il programma va in crash è colpa dell'utente — sul mio PC funziona"

Questi atteggiamenti sono incompatibili con lo sviluppo in team: documentazione, test e manutenibilità del codice non sono un "di più" opzionale, ma parte integrante di un lavoro professionale, tanto quanto scrivere codice che funziona al primo colpo sulla propria macchina.

3.8 Cos'è, allora, l'Ingegneria del Software

DEFINIZIONE DI LAVORO L'**Ingegneria del Software** è una disciplina ingegneristica che si occupa di **tutti** gli aspetti della produzione del software.

La componente "Ingegneria"

- Uso appropriato di modelli, metodi e strumenti
- Soluzioni innovative quando quelle esistenti non bastano
- Rispetto dei vincoli e uso efficiente delle risorse

La componente "Produzione"

- Scrivere codice è **solo una** delle attività tecniche necessarie
- Esistono anche attività **non tecniche** (gestione, comunicazione, organizzazione) altrettanto determinanti

Secondo **Tsui e Karam**, l'Ingegneria del Software è un settore ampio, che tocca tutti gli aspetti dello sviluppo e del supporto di un sistema software, includendo in particolare:

- Processi tecnici e processi aziendali
- Metodologie e tecniche specifiche
- Caratterizzazione dei prodotti e metriche per misure quantitative
- Abilità delle persone e lavoro di squadra

- Coordinamento e gestione dei progetti

3.9 Conclusioni della lezione

- La produzione del software è **intrinsecamente complessa** e comporta rischi economici significativi
- I fattori chiave di successo/fallimento non sono solo tecnici, ma **anche e soprattutto umani**
- Gestire queste sfide richiede una **metodologia strutturata**, fondata su un approccio ingegneristico
- Ogni progetto è unico: **non esiste una soluzione universale** valida per tutte le sfide

Riferimento del corso: I. Sommerville, *Ingegneria del Software*, 10^a ed., Pearson — Capitolo 1.

4 · Riepilogo per l'esame

Mappa concettuale in breve

Problema → per essere risolto bene serve capire **requisiti** (funzionali = cosa fa; non funzionali = come lo fa, misurati su scala) e **vincoli di progettazione** (piattaforma, tecnologie, integrazione, dati, tempi).

Processo → comprensione → progettazione → implementazione → test, in teoria lineare, in pratica **iterativo**. Per gestire tempi/costi si stima l'**effort** scomponendo il progetto in task (project breakdown).

Programma → **Sistema** → crescendo in funzionalità, interfacce, utenti, dati e connessioni, un sistema richiede un **team**, non più una persona sola. La complessità si gestisce con *divide et impera*, strumenti migliori, processo e metodologia migliori.

Comunicazione → con n persone i percorsi di comunicazione crescono come $n(n-1)/2$: servono **strutture organizzative**.

Perché serve l'Ingegneria del Software → la "crisi del software" (NATO 1968) e gli studi successivi (Chaos Report, Capers Jones) mostrano che i progetti falliscono soprattutto per **problemi di requisiti, comunicazione e organizzazione**, non (solo) per errori tecnici. La **legge di Brooks** ne è la sintesi più famosa: aggiungere persone a un progetto in ritardo lo ritarda ulteriormente, perché il costo di comunicazione supera il beneficio.

Domande di autoverifica

1. Perché un enunciato di problema non è mai sufficiente da solo per iniziare a programmare?

→ Perché lascia indefiniti aspetti come formato dei dati, criterio di ordinamento/elaborazione, gestione di errori e casi speciali, performance richieste e usabilità: tutte informazioni che vanno esplicitate come requisiti prima di progettare.

2. Qual è la differenza principale, anche nel modo in cui si misurano, tra requisiti funzionali e non funzionali?

→ I funzionali si misurano in modo booleano (soddisfatto/non soddisfatto: descrivono cosa deve fare il software); i non funzionali si misurano su una scala continua (descrivono quanto bene una caratteristica — es. il tempo di risposta — è soddisfatta).

3. Un vincolo di progettazione è sempre uguale a una scelta di progettazione?

→ No: un vincolo è imposto dall'esterno (dal cliente, dal contesto) e va rispettato; una scelta di progettazione è decisa liberamente dal team e può essere rivista.

4. Perché il processo di sviluppo reale è iterativo e non lineare come quello "ideale"?

→ Perché problemi scoperti durante il test o l'implementazione spesso costringono a tornare indietro alla progettazione o persino a rivedere la comprensione iniziale del problema.

5. Cosa cambia, oltre alla dimensione, passando da un programma a un sistema software?

→ Cambiano qualitativamente: numero e varietà di funzionalità, varietà di interfacce (interne ed esterne), numero e tipi di utenti, quantità e tipi di dati/strutture dati, connessioni con altri sistemi, e — di conseguenza — serve un team invece di una sola persona.

6. Quali sono i tre modi complementari per gestire la complessità di un sistema?

→ Divide et impera (decomposizione, modularizzazione, separazione delle responsabilità, iterazioni incrementali); strumenti migliori (DB, IDE, documentazione e test automatizzati, version control); miglioramento del processo/metodologia (coordinamento delle persone, lavoro di squadra).

7. Cosa deve specificare, oltre all'elenco delle attività, un vero processo di produzione del software?

→ La sequenza delle attività, i loro input/output, i partecipanti, le precondizioni per l'inizio e le postcondizioni per il completamento di ciascuna attività.

8. Perché aumentare il numero di persone in un team non riduce il tempo di sviluppo in modo proporzionale?

→ Perché il numero di percorsi di comunicazione cresce come $n(n-1)/2$, quindi con il quadrato del numero di persone: il costo di coordinamento cresce più velocemente del lavoro utile aggiunto da ogni nuova persona.

9. Cosa afferma la legge di Brooks e da quale assunzione sbagliata nasce il problema che descrive?

→ Afferma che aggiungere personale a un progetto software già in ritardo lo ritarda ulteriormente. Nasce dall'assunzione implicita che tempo e persone siano intercambiabili (mesi-uomo), che ignora sia l'indivisibilità di certe attività sia il costo crescente di comunicazione.

10. Secondo il Chaos Report e lo studio di Capers Jones, i progetti falliscono soprattutto per errori di codifica?

→ No: pur essendo gli errori di codifica i più frequenti in percentuale (~38%), le cause di fallimento più ricorrenti nei progetti reali riguardano requisiti incompleti/mutevoli e mancanza di comunicazione con gli utenti; inoltre gli errori nei requisiti, pur meno frequenti, sono spesso i più costosi da correggere perché scoperti tardi nel ciclo di vita.

11. Qual è la definizione di Ingegneria del Software vista a lezione, e cosa comprende secondo Tsui e Karam?

→ È la disciplina ingegneristica che si occupa di tutti gli aspetti della produzione del software (uso di modelli/metodi/strumenti, soluzioni innovative, rispetto di vincoli e risorse — non solo scrivere codice, ma anche attività non tecniche). Per Tsui e Karam comprende: processi tecnici e aziendali, metodologie e tecniche, metriche/caratterizzazione dei prodotti, abilità delle persone e lavoro di squadra, coordinamento e gestione dei progetti.